SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

QUESTION 1 – AI-POWERED DRONE FOR EMERGENCY MEDICINE DELIVERY

Scenario Understanding

A government uses AI-powered drones to deliver emergency medicines in a flood-affected region. Paths may become inaccessible, weather changes movement cost, and the drone receives only partial real-time updates. It must select a route with minimum delay while managing risk. This is a dynamic path-planning problem in which distance alone is not enough: travel time, energy, weather, restrictions, and safe landing or return options matter.

Key Issues

- Sudden blocked routes caused by floodwater, debris, or restricted airspace.
- Variable edge costs due to wind, rain, terrain, and changing travel time.
- Incomplete or delayed observations and uncertain map information.
- Trade-off among delivery delay, risk, energy, and reliability.
- Limited computation time and battery; strict safety constraints.

i. Greedy Best-First Search: Behavior, Strengths and Weaknesses

Greedy Best-First Search (GBFS) selects the frontier node with the smallest heuristic h(n). If h(n) is straight-line distance to the recipient, it repeatedly expands the location that appears closest to the goal. It does not account for the accumulated route cost g(n).

1. Represent candidate waypoints as nodes and possible movements as edges.
2. Calculate straight-line distance from each frontier waypoint to the destination.
3. Expand the waypoint with the smallest h(n), adding its feasible neighbors.
4. If new information reports a blocked path, invalidate that edge and replan from the current state.

Strengths: it can produce a promising route quickly; its ranking is simple; and it may be useful when a fast provisional route is needed.

Weaknesses: it may select a nearby-looking but costly detour, ignore costs already incurred, and perform badly when distance is a poor predictor of actual travel time. It is not generally optimal. With uncertain updates, a planned route can quickly become obsolete. GBFS therefore cannot serve as the safety mechanism.

ii. A* Search: Balancing Cost and Heuristic

A* evaluates each node using f(n) = g(n) + h(n). Here, g(n) is the accumulated cost from the start, while h(n) estimates the remaining cost to the goal. The node with the lowest estimated total cost is expanded.

- Model edges using the selected objective, such as estimated travel time; include weather and terrain effects in the cost.
- Use g(n) to account for route cost already accumulated.
- Use h(n) to estimate remaining cost. For a time objective, distance must be converted into a defensible lower-bound time estimate.
- Update best known costs and parent links as less costly paths are discovered.

- Validate a proposed route against current restrictions and safety rules before executing it.

For a static graph with nonnegative costs, an admissible heuristic and appropriate graph-search handling, A* can return an optimal route for the modeled objective. Its main limitations are memory use and potentially expensive computation.

iii. Impact of Heuristic Accuracy

For GBFS, h(n) directly controls node selection. A misleading heuristic can direct the drone toward an obstacle, dead end, or expensive path. A better estimate may speed the search but does not guarantee optimality. For A*, a useful admissible heuristic can reduce unnecessary expansions while retaining optimality under the algorithm's assumptions. If h(n) overestimates, the optimality guarantee may be lost.

Straight-line distance is not automatically a lower bound on time or risk. Wind, speed limits, detours, and restrictions must be considered. Heuristics should align with the objective, and hard safety constraints must be applied independently.

iv. Effect of Dynamic Blocked Paths

- Mark blocked edges unavailable immediately when credible updates arrive.
- Update travel costs when weather or wind changes.
- Replan from the drone's current state if the active route becomes infeasible or unsafe.
- Repeated fresh searches can consume time; stale information can also create unsafe decisions.
- Incremental methods such as D* Lite or Lifelong Planning A* can reuse prior search work in repeated replanning settings.
- Use event-triggered updates, battery reserves, contingency procedures, and communication-loss policies.

v. Recommendation

Recommend A* as the baseline planner when its cost model and compute budget are suitable. In highly dynamic operations, consider an incremental replanner such as D* Lite or Lifelong Planning A*. GBFS may supply a rapid provisional suggestion, but should not be the sole flight planner. First enforce hard constraints for no-fly zones, weather limits, collision avoidance, energy reserve, and safe landing; then optimize the approved objective. Safety takes priority over minimum delay.

Conclusion

A* offers a stronger least-cost foundation than GBFS because it considers both cost already paid and estimated future cost. Live sensing, rapid replanning, route validation, and fail-safe operation remain essential.

QUESTION 2 – SMART-CITY TRAFFIC SIGNAL OPTIMIZATION

Scenario Understanding and Optimization Formulation

A smart city adjusts signal timings across hundreds of intersections to reduce total waiting time, fuel consumption, and congestion. The system operates under changing traffic flow and has a huge search space with possible local optima.

Let a candidate solution S encode each intersection's phase sequence, green-time splits, offsets, and cycle lengths over a planning horizon. Define an objective:

Minimize $J(S) = \alpha W(S) + \beta F(S) + \gamma C(S)$

where W is total vehicle waiting time, F is estimated fuel consumption, C is a congestion measure, and α, β, γ are approved weights that express priorities and normalize units. Subject to: safe minimum/maximum green times, pedestrian crossing time, clearance intervals, phase compatibility, coordination constraints, and legal signal rules. The objective can also include a penalty for excessive timing changes.

Why Traditional Search Is Inefficient

If each of many intersections has numerous timing combinations, the joint state space grows combinatorially. Exhaustive uninformed search evaluates too many candidates. The state is also nonstationary: traffic counts and queues change while the search runs. A model-based search can optimize against outdated estimates, while simulation of each candidate may be expensive.

i. Hill Climbing in This Scenario

5. Start with a feasible signal plan, such as current timings or a coordinated baseline.
6. Define neighboring plans by small changes to one or more timings or offsets.
7. Evaluate the objective using recent traffic measurements or a simulator.
8. Move to a neighbor with a better score and repeat until no tested neighbor improves the result.
9. Apply only validated timing changes, then measure performance and optimize again.

Hill Climbing is easy to implement, requires modest memory, and can improve a reasonable initial plan quickly. However, it is local: it may stop at a local optimum, plateau, or ridge. Its result depends on initialization, neighborhood definition, and the quality of the traffic model.

ii. Local Maxima, Plateaus and Ridges

- Local maximum (or minimum in a cost formulation): a plan is better than nearby plans but worse than a distant plan. For example, improving one intersection alone may worsen queues downstream, hiding a better coordinated change.
- Plateau: many nearby timing plans have nearly equal scores. The algorithm may stop or wander without meaningful progress, especially with noisy traffic estimates.
- Ridge: progress requires a coordinated sequence of changes across multiple signals. Changing one green split at a time may not improve the score, even though coordinated offset changes would.

iii. Simulated Annealing or Genetic Algorithms

Simulated Annealing (SA) sometimes accepts a worse move to escape a local optimum. For a cost increase Δ, it may accept the move with probability $\exp(-\Delta/T)$, where T is a temperature parameter. Start with higher T for exploration and gradually cool. SA is comparatively compact and can work

with a well-designed neighborhood, but its performance depends on cooling schedule and evaluation cost.

A Genetic Algorithm (GA) maintains a population of candidate timing plans. It evaluates fitness, selects better candidates, recombines timing-plan segments, and applies mutations. Population diversity and crossover can explore different regions of the search space. GA is naturally parallelizable, but can require many evaluations and may converge prematurely or produce infeasible candidates unless constraints are carefully handled.

iv. Recommendation: Hybrid Adaptive Optimization

Recommend a rolling-horizon hybrid: use a feasible coordinated baseline, then apply a GA or SA within a bounded computation budget, followed by local refinement and safety/constraint validation. For large city networks, optimize regions or corridors with coordination between boundaries rather than independently tuning every light.

- Scalability: parallel candidate evaluation and corridor/region decomposition can distribute work.
- Adaptability: continuously ingest traffic data and re-optimize at scheduled intervals or after major incidents.
- Robustness: evaluate candidates against multiple traffic scenarios and penalize unstable or brittle plans.
- Safety: enforce pedestrian, clearance, minimum/maximum green, and conflict constraints as hard requirements.
- Real-time operation: maintain a safe fallback plan and apply changes gradually to avoid oscillation.

GA is a reasonable primary optimizer when many timing decisions interact and parallel compute is available; SA may be preferable where memory and implementation simplicity are priorities. The practical choice should be confirmed through simulation, stress testing, and field trials.

Conclusion

Hill Climbing is fast but can become trapped. A bounded, adaptive metaheuristic—often GA with local improvement, or SA for lower overhead—offers stronger exploration, provided safety constraints and real-time limits are enforced.

QUESTION 3 – AUTONOMOUS MARS ROVER IN UNKNOWN TERRAIN

Scenario Understanding

A Mars rover must travel safely, avoid craters and rocks, and collect samples despite limited sensing and no complete map. It updates its knowledge as it moves. This requires planning while learning about the environment.

i. Why an Online Search Agent Is Needed

Offline search assumes the relevant state space and action outcomes are sufficiently known before execution. The rover cannot rely on a complete map or perfect transition information. An online search agent interleaves action, observation, model update, and planning. It can discover an obstacle, update its map, and choose a new action based on evidence.

ii. Partial Observability and Dynamic Challenges

- Limited sensor range and noise may hide hazards or make terrain classification uncertain.
- Communication delay or outages prevent continuous remote guidance.
- Terrain traversability, slope, wheel slip, and energy use may be uncertain.
- The rover must balance exploration for information against immediate progress and safety.
- A hazard may be static, but knowledge changes as it becomes visible; dust, lighting, or environmental conditions can also alter observations.
- Battery, thermal limits, time, and sample-handling constraints limit exploration.

A partially observable system can maintain a belief state: a representation of possible rover locations or terrain conditions with confidence estimates, rather than assuming every observation is certain.

iii. Building and Updating the Internal Model

10. Initialize a local map from landing-site data and onboard sensors, including uncertainty.
11. At each step, collect camera, range, inertial, wheel, and other available observations.
12. Estimate rover pose and register new observations to the map.
13. Classify cells or terrain patches as traversable, hazardous, or unknown, with confidence values.
14. Update movement-cost estimates using slope, roughness, slip, energy, and risk.
15. Plan a safe next action or short horizon, execute it, compare expected and observed outcomes, and update the model again.

Unknown terrain should not automatically be treated as safe. The planner can assign conservative costs, require additional observation, or prohibit travel when uncertainty exceeds a safety threshold.

iv. Online Search Compared with Uninformed and Informed Search

Uninformed methods such as breadth-first search or uniform-cost search do not use goal-directed heuristic information. They can be systematic but may explore many states and generally assume a sufficiently known graph. Informed methods such as Greedy Best-First Search and A* use a heuristic to focus exploration. A* can account for known route costs and estimated remaining cost, but standard versions still assume a modeled graph.

Online search focuses on decisions under incomplete knowledge, repeatedly acting and learning. It can use informed or uninformed planning internally, but must also handle unknown transitions and observations. Thus, “online” describes when learning and planning occur, while “informed/uninformed” describes how search is guided.

v. Recommended Approach

Recommend a receding-horizon, risk-aware online planner: use onboard mapping and state estimation, conservative traversability costs, and A* or another suitable local planner over the currently known safe map. Replan after meaningful observations. Use exploration actions only when expected information value justifies the risk and resource cost.

- Adaptability: update maps and costs from each new observation.
- Safety: maintain hazard margins, uncertainty thresholds, speed limits, and safe stopping/retreat options.
- Efficiency: plan locally or hierarchically instead of searching the entire planet-scale map.
- Resilience: preserve a safe-mode behavior if sensors, localization, or communication degrade.
- Verification: test perception and planning across simulated terrain and fault scenarios.

Conclusion

An online approach is essential because the rover learns the terrain during travel. A local, risk-aware planner supported by uncertainty-aware mapping and strict safety rules provides a practical balance between exploration, progress, and survival.

QUESTION 4 – UNIVERSITY EXAM TIMETABLE GENERATION AS A CSP

Scenario Understanding

The university must assign examinations to times and halls while avoiding student conflicts, respecting hall capacity, precedence requirements, faculty availability, and accommodations. This is a Constraint Satisfaction Problem (CSP).

CSP Formulation: Variables, Domains and Constraints

A CSP is defined by variables X, domains D, and constraints C. One clear model uses one variable per exam: Xc = (time slot, hall) assigned to course c. Alternatively, separate time and hall variables can be used for clearer propagation.

i. Variables, Domains and Constraints

- Variables: for each exam c, create Tc for its time slot and Hc for its hall (or a combined assignment variable). Include additional variables for required accessibility resources or invigilator assignments if those are part of the timetable.
- Time domain: all approved exam slots that satisfy the exam duration and calendar.
- Hall domain: available halls with sufficient capacity, required accessibility, equipment, and permitted time availability.
- Student non-overlap: two exams sharing any student must be assigned to different time slots.
- Hall conflict: a hall cannot host two exams whose scheduled durations overlap.
- Capacity: assigned hall capacity must meet enrollment plus required accommodation needs.
- Precedence: if exam A must occur before exam B, enforce time(A) < time(B).
- Faculty availability: exams requiring a faculty member or invigilator must be placed in available periods; avoid overlapping duties.
- Accommodation: provide accessible venues, extra duration, breaks, or separate conditions where officially required.
- Institutional rules: include exam spacing, daily load limits, room setup time, and other policy requirements.

A solution is a complete assignment that satisfies every hard constraint. Preferences such as avoiding early slots or distributing workload can be represented as soft constraints and optimized only after feasibility is secured.

ii. Backtracking Search

16. Begin with no exam assignments.
17. Select an unassigned variable, preferably the most constrained exam.
18. Choose a candidate time and hall from its current domain.
19. Check whether the assignment conflicts with any existing assignment.
20. If consistent, assign it and continue recursively.
21. If no value works, undo the latest assignment and try another value.
22. Stop when all exams are assigned or report that the current model has no solution.

Backtracking is complete for a finite CSP when implemented systematically, but naïve variable/value ordering can lead to extensive search.

iii. Constraint Propagation and Forward Checking

After assigning an exam, forward checking removes values from neighboring unassigned variables that would violate the new assignment. For example, assigning a course to slot 2 removes slot 2 from the domains of exams sharing students. A hall assignment also removes overlapping uses of that hall.

- Detects dead ends earlier than waiting until a full timetable is built.
- Reduces the domains and number of choices considered.
- Can be combined with Maintaining Arc Consistency (MAC) for stronger propagation, though it adds overhead.
- Use Minimum Remaining Values (MRV) to select the exam with the smallest legal domain, Degree Heuristic to break ties, and Least Constraining Value (LCV) to choose a value that blocks the fewest future options.

iv. Real-World Challenges and Scalable Strategy

- Large conflict graphs: popular courses may share many students and be highly constrained.
- Changing enrollment, room outages, staff availability, and accommodation requests require model updates.
- Some requirements conflict, so the institution must distinguish non-negotiable hard constraints from preferences.
- Duration, setup buffers, shared resources, and accessibility details increase model complexity.
- Data quality and privacy matter when using student enrollment and accommodation records.

Recommend CSP backtracking with MRV, degree heuristic, LCV, and forward checking or MAC as a strong baseline. For large instances, exploit structure: construct the exam-conflict graph, decompose by campuses or time blocks where valid, use constraint programming/SAT/ILP solvers, and apply local repair after changes. Use weighted soft constraints for quality, but never relax critical safety, legal, or approved accommodation requirements without formal authorization.

Conclusion

CSP modeling makes timetable rules explicit and verifiable. Intelligent ordering and propagation prune infeasible choices early; solver technology and structured decomposition support larger instances.

QUESTION 5 – STRATEGIC GAME AI: MINIMAX AND ALPHA-BETA PRUNING

Scenario Understanding

A game AI must compete with human players, choose actions under strict time limits, and reason about many possible moves. A large branching factor and deep game tree make exhaustive search impractical.

i. How Minimax Models the Problem

Minimax models a competitive, typically two-player, zero-sum game. Nodes represent game states and edges represent legal actions. The AI (MAX) seeks to maximize utility, while the opponent (MIN) is assumed to choose moves that minimize MAX's utility.

Generate legal moves from the current state.

Expand possible future states to a chosen depth or terminal condition.

Score terminal states using actual win/loss/draw utility.

At MAX nodes, propagate the maximum child value; at MIN nodes, propagate the minimum.

Choose the root action with the highest backed-up value.

Minimax is appropriate when the game is sufficiently deterministic and adversarial. Games with randomness or hidden information require extensions such as expectiminimax, belief-state reasoning, or sampling.

ii. Computational Challenges

With branching factor b and search depth d, basic minimax explores O(b^d) nodes. The number of states can grow rapidly, and game evaluation, legal move generation, and state copying add cost. A fixed time budget may prevent reaching the desired depth. A shallow search can miss delayed tactics, while a deep search may return too late to act.

Large branching factor and long tactical sequences.

Repeated states caused by different move orders.

Uncertainty about the opponent's actual strategy.

Strict frame or turn deadlines and variable hardware load.

Evaluation-function errors and horizon effects, where a danger lies just beyond the search depth.

iii. Alpha-Beta Pruning with Detailed Reasoning

Alpha-Beta pruning returns the same minimax decision as full minimax when applied correctly, but avoids searching branches that cannot affect the final choice. Alpha (α) is MAX's best guaranteed value so far; beta (β) is MIN's best guaranteed value so far.

Initialize $\alpha = -\infty$ and $\beta = +\infty$ at the root.

At a MAX node, update α when a better child value is found.

At a MIN node, update β when a smaller child value is found.

If $\alpha \geq \beta$, stop examining the remaining children of that node: they cannot improve the decision for the ancestor.

Return the backed-up value and continue until the root action is selected.

Example reasoning: suppose MAX already has an option worth 8, so $\alpha = 8$. While evaluating another option, a MIN opponent can already force that option to 6 ($\beta = 6$). Since $\alpha \geq \beta$, MAX would never prefer that branch over its existing value 8, so its remaining children can be pruned.

In the worst case, alpha-beta remains O(b^d). With ideal move ordering, the effective search can approach O(b^(d/2)), allowing deeper analysis. Move ordering is therefore important: examine likely strong moves first, using prior best moves, transposition-table hints, captures, threats, or domain-specific priorities.

iv. Evaluation Function Design

For a nonterminal state s, use a bounded, empirically tuned evaluation such as:

E(s) = w1·ResourceAdvantage + w2·Territory/Control + w3·UnitStrength + w4·ObjectiveProgress + w5·Mobility − w6·Exposure/Risk − w7·ResourceDeficit

Each term should be normalized to comparable ranges and evaluated from the AI player's perspective. Weights depend on game design and should be tuned through self-play and testing.

Resource advantage: economy, energy, or production capacity available for future actions.

Territory/control: strategically useful map regions and access routes.

Unit strength: surviving units, capabilities, health, and force composition.

Objective progress: capture points, mission completion, or strategic goals.

Mobility: available legal actions and positional flexibility.

Exposure/risk: vulnerability to counterattack, supply disruption, or losing key units.

Resource deficit and opportunity cost: upkeep, cooldowns, or overextension.

Use terminal scores that dominate ordinary heuristic values so that a forced win is preferred to a merely favorable nonterminal position. Avoid double-counting correlated features and test for unintended incentives.

v. Recommended Real-Time Strategy

Recommend iterative deepening minimax with alpha-beta pruning, strong move ordering, a transposition table, and a deadline-aware search controller.

Iterative deepening: search depth 1, then 2, and so on; always retain the best move from the last completed depth.

Alpha-beta: prune branches that cannot change the minimax decision.

Move ordering: prioritize promising actions to increase pruning.

Transposition table: cache evaluated states with correct bound types and depth information.

Quiescence or selective extensions: continue tactically unstable lines where appropriate, while guarding against unbounded growth.

Time management: reserve time for selecting and validating an action; stop safely before the hard deadline.

Evaluation: tune features with self-play, benchmarks, and adversarial test cases.

Opponent adaptation: track behavior cautiously; do not assume the opponent always follows a predictable pattern.

This approach does not guarantee a globally optimal move under a hard depth/time limit, but it gives strong decisions while retaining a valid fallback. If the game has simultaneous actions, randomness, or hidden state, adapt the model rather than applying ordinary minimax blindly.

Conclusion

Minimax provides the adversarial decision framework; alpha-beta pruning makes it more practical by avoiding irrelevant branches. Iterative deepening, move ordering, caching, evaluation quality, and strict time management offer a balanced real-time design.